SkillSync – Peer Learning & Mentor Matching Platform

Domain: Professional mentorship platform.

1. System Overview

SkillSync is a **multi-user platform** that connects learners with mentors and enables peer learning. Users can search for mentors, book mentoring sessions, join learning groups, and review mentors after sessions.

The platform must support:

- multi-role users
- real-time mentor discovery
- session booking and scheduling
- peer learning groups
- notifications and reminders
- scalable microservices architecture

The backend system will be built using **Spring Boot microservices architecture** with **event-driven communication via RabbitMQ**.

---

2. Business Goals

The platform aims to:

- help learners find mentors easily
- allow experts to monetize their skills
- enable peer learning communities
- provide reliable scheduling and communication
- maintain secure authentication and authorization

---

3. Key Actors

1. Learner

A user who wants to learn new skills.

Capabilities:

- search mentors
- request mentoring sessions
- join study groups
- rate mentors

---

2. Mentor

An expert who teaches skills.

Capabilities:

- create mentor profile
- list skills
- set availability
- accept/reject sessions
- receive ratings

---

3. Admin

Platform administrator.

Capabilities:

- manage users
- approve mentors
- moderate groups
- monitor platform activity

---

4. Functional Requirements

4.1 Authentication & Authorization

Users must be able to register and login securely.

Features:

- user registration
- login with credentials
- JWT token generation
- role-based access control
- secure API access

Supported roles:

ROLE_LEARNER
ROLE_MENTOR
ROLE_ADMIN

4.2 User Profile Management

Users maintain profiles containing personal and professional details.

Features:

- create profile
- update profile
- add skills
- view profile

- upload profile image

4.3 Mentor Onboarding

Mentors must apply before becoming active.

Workflow:

1 User registers
2 User applies to become mentor
3 Admin reviews mentor request
4 Mentor profile activated

Features:

- mentor application
- mentor profile creation
- mentor skill tagging
- mentor experience details

4.4 Skill Catalog

Platform maintains a centralized skill list.

Examples:

- Java

- Spring Boot
- Data Structures
- Machine Learning

Features:

- create skills (admin)
- view skills
- tag skills to mentors

4.5 Mentor Discovery

Learners should easily discover mentors.

Filters include:

- skill
- rating
- experience
- price
- availability

Features:

- search mentors
- view mentor profile
- view mentor ratings

4.6 Session Booking

Learners can schedule mentoring sessions.

Workflow:

1 learner selects mentor
2 learner requests session
3 mentor accepts or rejects
4 session scheduled

Session states:

REQUESTED
ACCEPTED
REJECTED
COMPLETED
CANCELLED

4.7 Peer Learning Groups

Users can create study groups.

Features:

- create learning group
- join group
- leave group
- group discussions

Examples:

Spring Boot Beginners
DSA Interview Preparation
Machine Learning Study Circle

4.8 Reviews and Ratings

After sessions learners can review mentors.

Features:

- submit rating
- submit review
- calculate average mentor rating

4.9 Notifications

Users receive notifications for important events.

Examples:

- session booked
- session accepted
- session reminder
- mentor approved

Notifications are handled **asynchronously via RabbitMQ**.

---

5. Non Functional Requirements

Scalability

Microservices architecture allows independent scaling.

Security

Spring Security with JWT.

Fault tolerance

RabbitMQ ensures reliable event delivery.

Maintainability

Each service owns its business logic.

Performance

Service communication optimized using asynchronous messaging.

---

6. High Level Architecture (HLD)

System architecture:

```
Client (Web / Mobile)
      |
API Gateway (Spring Cloud Gateway)
      |
Service Discovery (Eureka)
      |
-----------------------------------------------
| Auth Service                                |
| User Service                                |
| Mentor Service                              |
| Skill Service                               |
| Session Service                             |
| Group Service                               |
| Review Service                              |
| Notification Service                        |
-----------------------------------------------
      |
    RabbitMQ
      |
Databases (one database per microservice)
```

---

7. Microservices Breakdown

7.1 Auth Service

Handles authentication.

Responsibilities:

- register user
- login
- generate JWT
- validate JWT

APIs

POST /auth/register
POST /auth/login
POST /auth/refresh

Database tables:

users
roles
user_roles

7.2 User Service

Handles user profiles.

Responsibilities:

- create profile
- update profile

- fetch user details

APIs

GET /users/{id}
PUT /users/{id}
GET /users

7.3 Mentor Service

Handles mentor specific logic.

Responsibilities:

- mentor application
- mentor profile
- mentor availability
- mentor search

APIs

POST /mentors/apply
GET /mentors
GET /mentors/{id}
PUT /mentors/{id}/availability

7.4 Skill Service

Handles platform skills.

Responsibilities:

- manage skills
- skill lookup

APIs

POST /skills
GET /skills
GET /skills/{id}

7.5 Session Service

Handles mentorship sessions.

Responsibilities:

- session request
- session acceptance
- session cancellation

APIs

POST /sessions
PUT /sessions/{id}/accept
PUT /sessions/{id}/reject
PUT /sessions/{id}/cancel
GET /sessions/user/{userId}

7.6 Group Service

Handles peer learning groups.

Responsibilities:

- create groups
- join groups
- manage group members

APIs

POST /groups
POST /groups/{id}/join
POST /groups/{id}/leave
GET /groups
GET /groups/{id}

7.7 Review Service

Handles mentor ratings.

Responsibilities:

- add review
- fetch mentor reviews
- calculate ratings

APIs

POST /reviews

GET /reviews/mentor/{mentorId}

7.8 Notification Service

Handles asynchronous notifications.

Responsibilities:

- send email
- send push notifications
- process events from RabbitMQ

---

8. Event Driven Architecture (RabbitMQ)

Events allow services to communicate without tight coupling.

Example event:

Session booked.

```
Session Service
    |
    | publish event
    |
RabbitMQ
    |
Notification Service
```

Event payload example:

```
SESSION_BOOKED
{
```

sessionId
mentorId
learnerId
sessionTime
}

Notification service sends:

- email
- push notification

---

9. Database Design (High Level)

User

id
name
email
password
role
created_at

Mentor

id
user_id
bio
experience
rating
hourly_rate

Skill

id

name
category

Mentor Skill

mentor_id
skill_id

Session

id
mentor_id
learner_id
session_date
status
created_at

Review

id
mentor_id
user_id
rating
comment
created_at

Group

id
name
description
created_by
created_at

Group Members

group_id
user_id
joined_at

---

10. Security Architecture

Authentication Flow

```
Client Login
    |
Auth Service validates credentials
    |
JWT Token generated
    |
Client sends token with each request
    |
API Gateway validates token
    |
Request routed to service
```

Authorization example

Only mentors can accept sessions
Only admins approve mentors
Only learners can book sessions

---

11. Deployment Architecture

Deployment can include:

API Gateway
Eureka Server
Config Server
Auth Service
User Service

Mentor Service
Session Service
Group Service
Review Service
Notification Service
RabbitMQ
MySQL/PostgreSQL

Each service runs independently.

---

12. Future Enhancements

Possible improvements:

- video mentoring integration
- AI mentor recommendation
- payment integration
- calendar sync

real-time chatnwnvwvklwekwl

SKILLSYNC

Peer Learning & Mentor Matching Platform

FRONTEND CASE STUDY — ANGULAR 18

Sprint 1 | 2025

1. Project Overview

SkillSync is a modern peer learning and mentor matching web platform. This document describes the Angular 18 frontend architecture, component design, routing, state management, and UX for each key screen.

Attribute	Detail
Framework	Angular 18 (Standalone Components)
Language	TypeScript 5.x
Styling	Angular Material 18 + Custom SCSS
State Management	NgRx SignalStore (Signals-based)
HTTP Client	Angular HttpClient + JWT Interceptor
Auth	JWT — memory storage + HttpOnly refresh cookie
Build Tool	Vite + Angular CLI 18
Real-time	RxJS WebSocketSubject (Notifications)
Testing	Jest (Unit) + Cypress (E2E)
Deployment	Docker (Nginx) + GitHub Actions CI/CD

1.1 User Roles

Role	Frontend Access
ROLE_LEARNER	Dashboard, Browse Mentors, Book Sessions, Join Groups, Submit Reviews
ROLE_MENTOR	Mentor Dashboard, Manage Availability, Accept/Reject Sessions, View Ratings
ROLE_ADMIN	Admin Console: User Management, Mentor Approvals, Platform Analytics

2. Angular 18 Project Structure

src/ — Feature-Based Folder Structure

```
src/
├── app/
│   ├── core/              # AuthService, Guards, Interceptors, ApiService
│   ├── shared/            # Reusable components, directives, pipes
│   ├── features/
│   │   ├── auth/          # Login, Register, Forgot Password
│   │   ├── dashboard/     # Learner & Mentor role-aware dashboards
```

```
│   │   ├── mentors/        # Browse, Profile, Mentor Discovery
│   │   ├── sessions/       # Book, Manage, Session History
│   │   ├── groups/         # Create, Join, Group Discussions
│   │   ├── reviews/        # Rate & Review mentors
│   │   ├── profile/        # Edit Profile, Skills management
│   │   └── admin/          # Admin Console panel
│   ├── layout/             # Shell, Navbar, Sidebar, Footer
│   ├── app.routes.ts       # Root routing with lazy loading
│   └── app.config.ts       # provideRouter, provideHttpClient...
├── assets/                 # Images, icons, fonts
├── environments/           # environment.ts / environment.prod.ts
└── styles/                 # Global SCSS, design tokens, Material theme
```

3. Routing Architecture

app.routes.ts — Lazy Loaded Routes with Guards

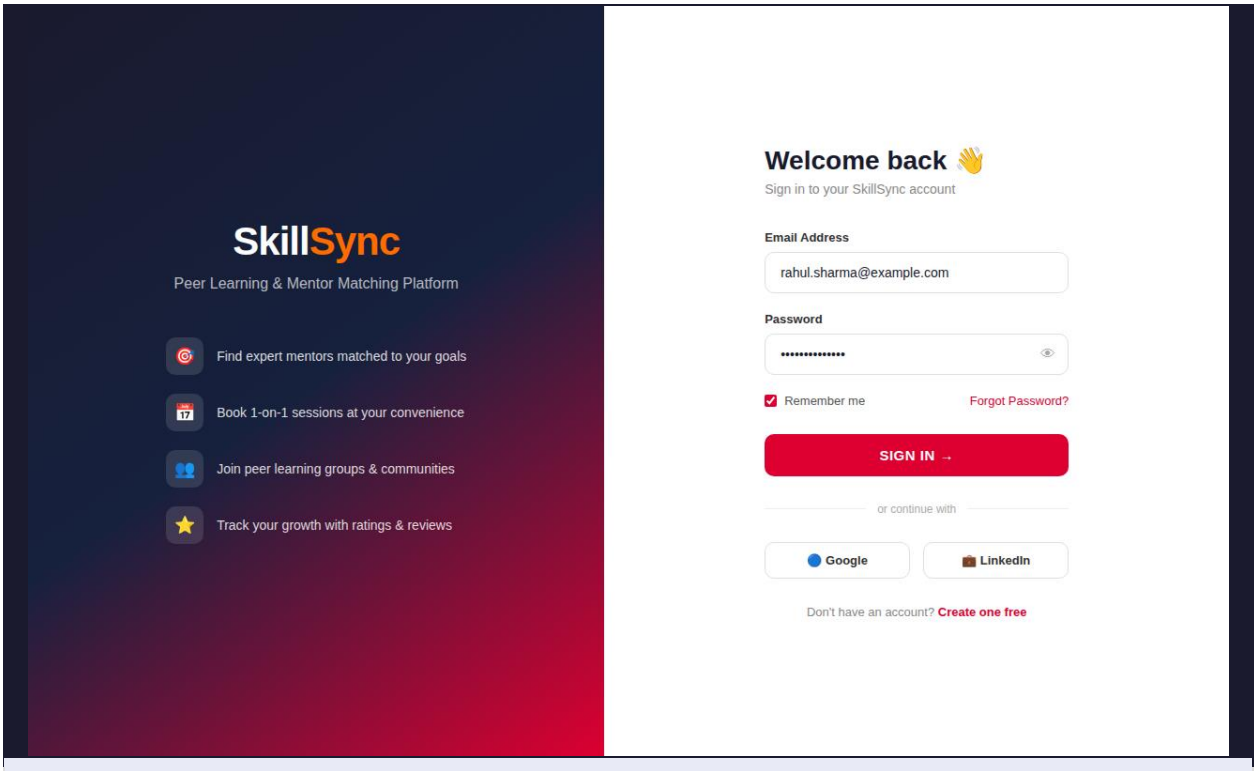
```
export const appRoutes: Routes = [
  { path: 'auth', loadChildren: () =>
      import('./features/auth/auth.routes').then(m => m.AUTH_ROUTES) },

  { path: '', component: ShellComponent, canActivate: [AuthGuard], children: [
    { path: 'dashboard',  loadComponent: () => import('./features/dashboard/...') },
    { path: 'mentors',    loadChildren: () => import('./features/mentors/...') },
    { path: 'sessions',   loadChildren: () => import('./features/sessions/...') },
    { path: 'groups',     loadChildren: () => import('./features/groups/...') },
    { path: 'profile',    loadComponent: () => import('./features/profile/...') },
    { path: 'admin',      loadChildren: () => import('./features/admin/...'),
      canActivate: [RoleGuard], data: { role: 'ROLE_ADMIN' } },
  ]},

  { path: '**', loadComponent: () => import('./shared/components/not-found/...') }
];
```

4. Application Screens & UI Design

Below are the actual rendered screenshots of the Angular 18 SkillSync web application. Each screen uses Angular Material components with a custom Angular red theme.

4.1 Authentication — Login Screen

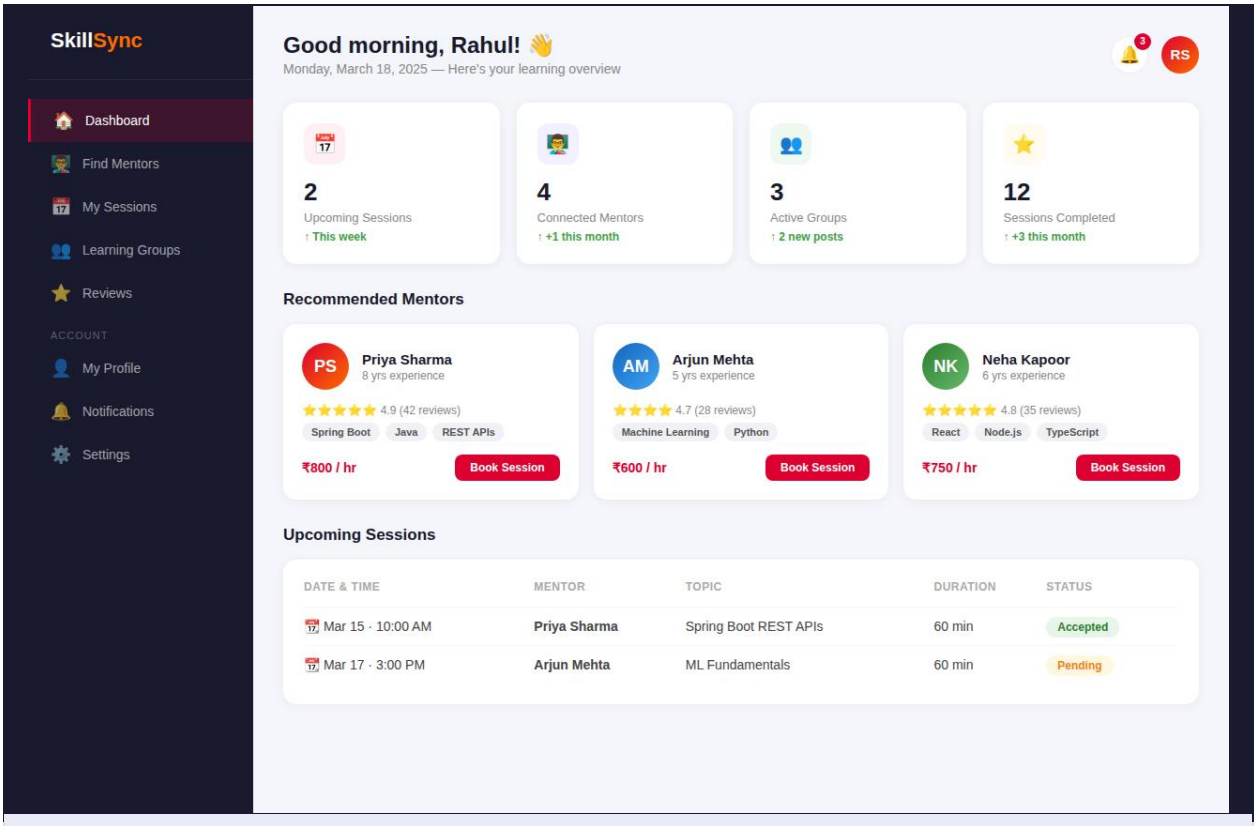
Reactive Forms with inline validation, JWT-based auth, and social login options (Google, LinkedIn). The split-panel layout highlights the platform value proposition.



Login Screen — Angular Reactive Forms + JWT Auth

4.2 Learner Dashboard

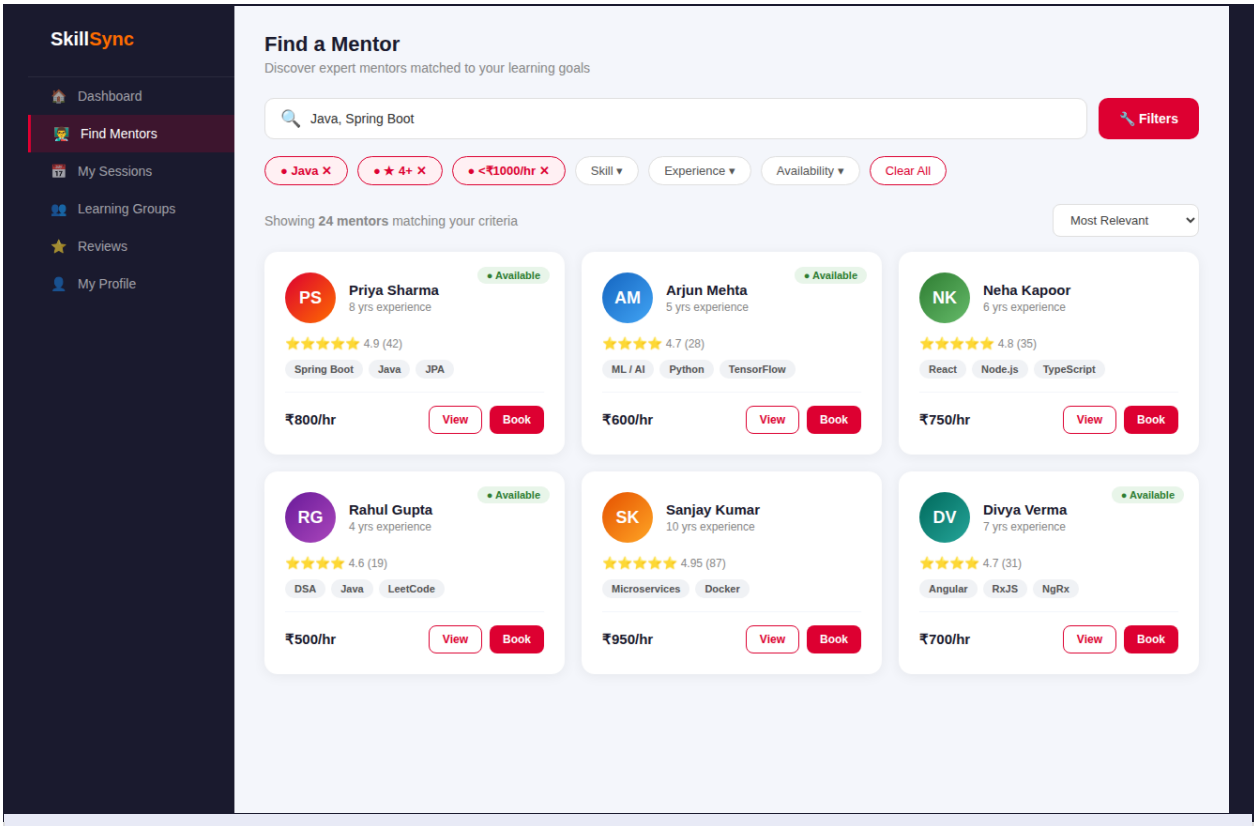
The role-aware dashboard uses Angular Signals to reactively display upcoming sessions, stat cards, recommended mentors, and quick actions. Navigation adapts based on user role.



Learner Dashboard — Stats, Recommended Mentors & Upcoming Sessions

4.3 Mentor Discovery & Search

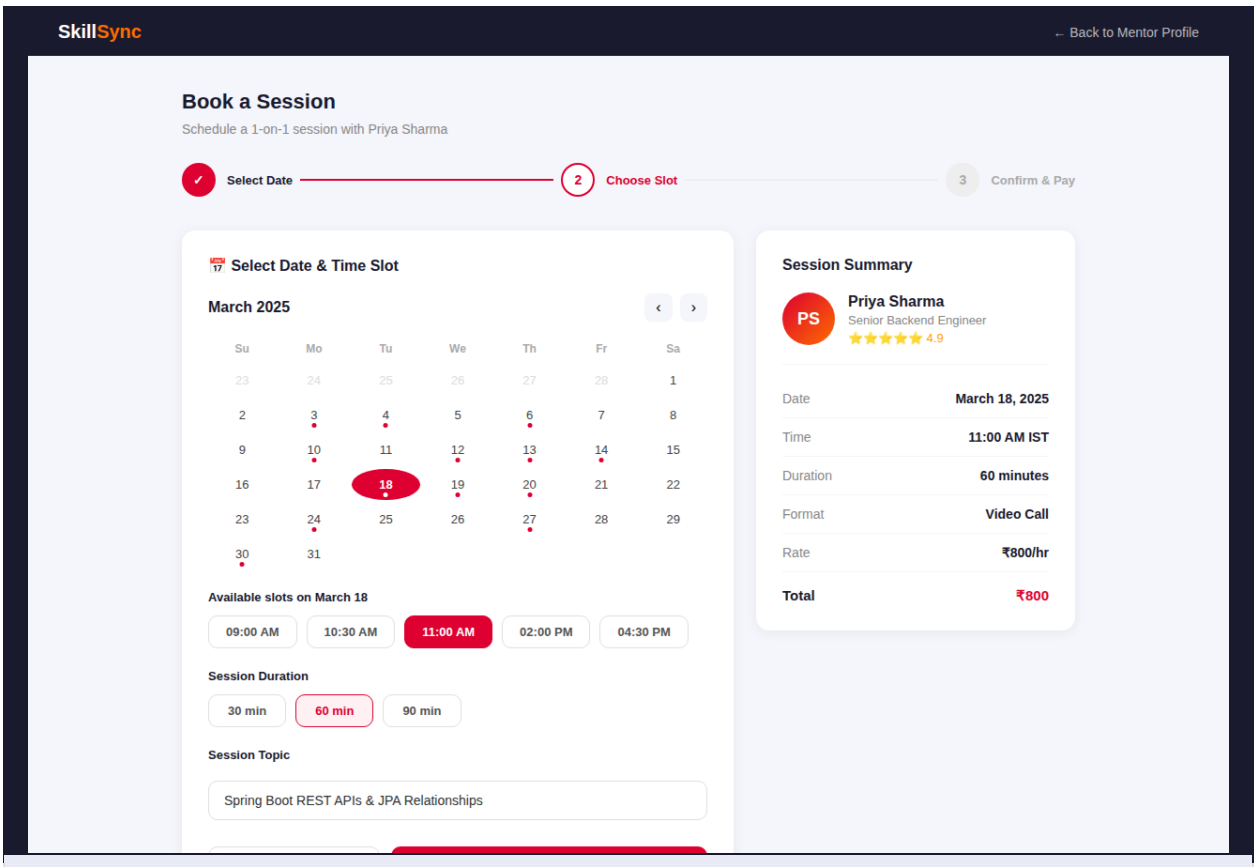
Learners can browse mentors with faceted filters (skill, rating, price, availability). Angular Signals drive reactive filter state. Debounced search uses RxJS operators.



Mentor Discovery — Filtered Search with Angular Signals

4.4 Session Booking Flow

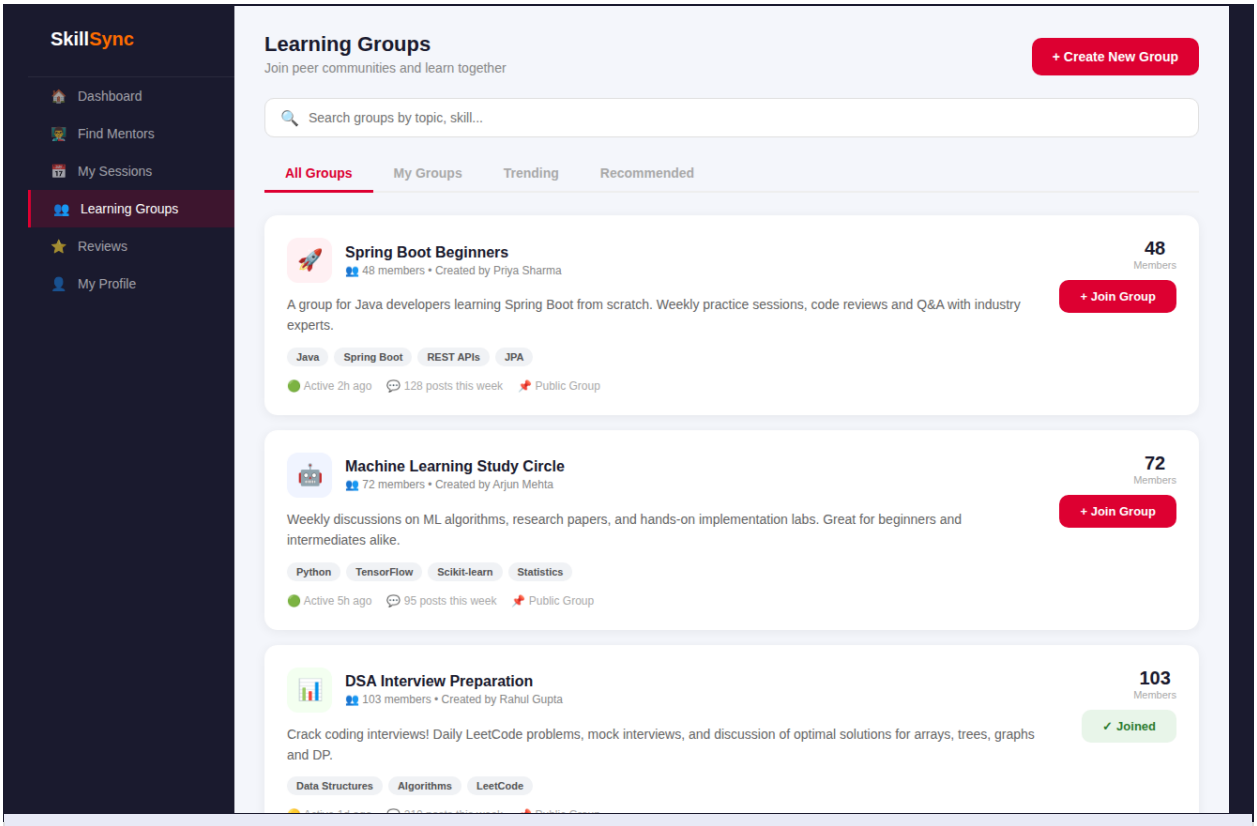
A multi-step CDK Stepper component guides the learner through: selecting a date, choosing a time slot, setting duration & topic, then confirming. The summary panel updates reactively.



Book a Session — Angular CDK Stepper with Interactive Calendar

4.5 Peer Learning Groups

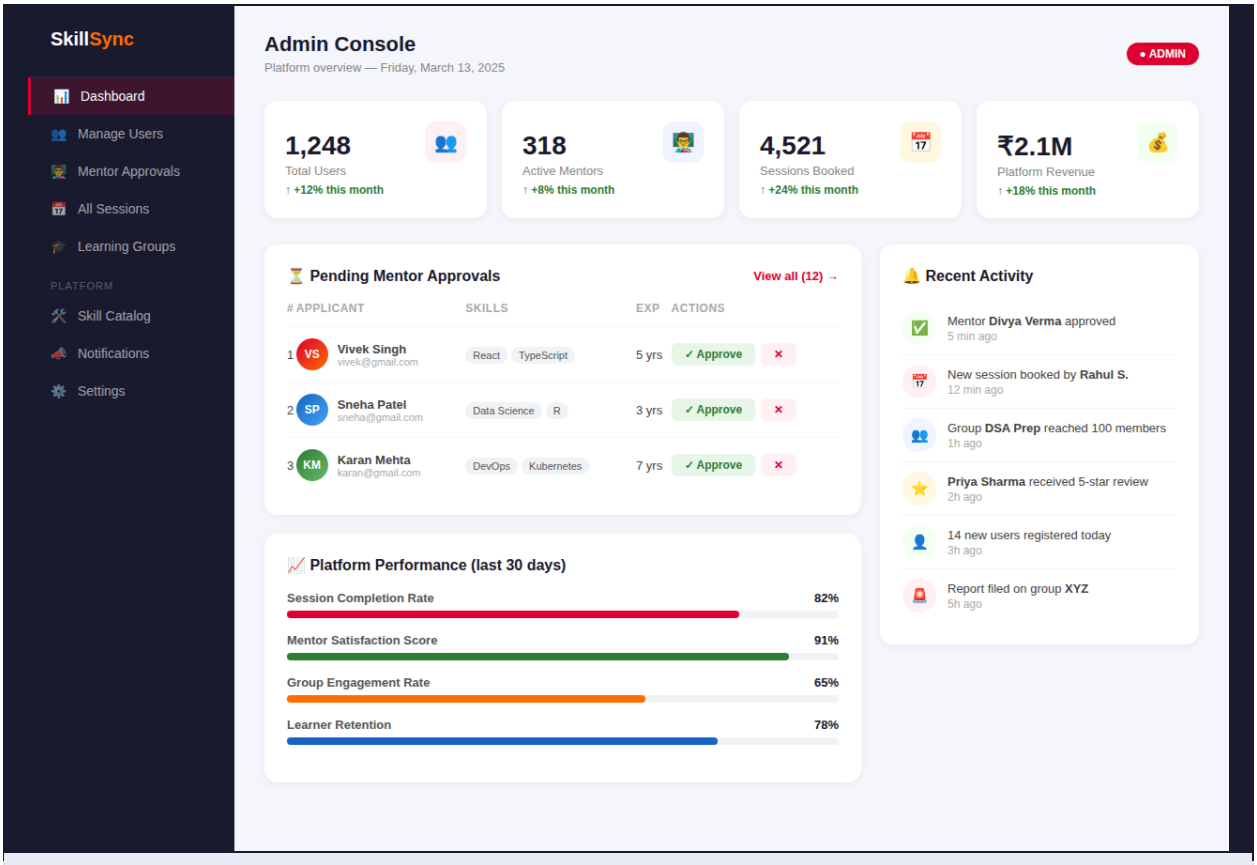
Users can discover, join, and create learning groups. Cards display member counts, activity status, and relevant skill tags. Joined groups show a distinct confirmation state.



Learning Groups — Browse, Join & Create Communities

4.6 Admin Console

The admin panel provides full platform oversight with KPI cards, pending mentor approval queue, performance progress bars, and a live activity feed.



Admin Console — Platform KPIs, Mentor Approvals & Activity Feed

5. State Management — Angular Signals + NgRx

auth.store.ts — NgRx SignalStore

```
export const AuthStore = signalStore(
  { providedIn: 'root' },
  withState<AuthState>({ user: null, token: null, isLoading: false }),
  withComputed(({ user }) => ({
    isAuthenticated: computed(() => !!user()),
    isAdmin:  computed(() => user()?.roles.includes('ROLE_ADMIN') ?? false),
    isMentor: computed(() => user()?.roles.includes('ROLE_MENTOR') ?? false),
  })),
  withMethods((store, authService = inject(AuthService)) => ({
    async login(credentials: LoginRequest) {
      patchState(store, { isLoading: true });
      const res = await authService.login(credentials);
      patchState(store, { user: res.user, token: res.token, isLoading: false });
    },
    logout() { patchState(store, { user: null, token: null }); }
  }))
);
```

6. API Integration

6.1 Frontend ↔ Backend API Mapping

Angular Service Method	Spring Boot Endpoint
AuthService.login()	POST /auth/login
AuthService.register()	POST /auth/register
MentorService.getAll()	GET /mentors
MentorService.search(filters)	GET /mentors?skill=&rating=&price=
SessionService.book()	POST /sessions
SessionService.accept(id)	PUT /sessions/{id}/accept
GroupService.getAll()	GET /groups
GroupService.join(id)	POST /groups/{id}/join
ReviewService.submit()	POST /reviews
NotificationService (WS)	WebSocket /ws/notifications

6.2 JWT Interceptor

```
jwt.interceptor.ts
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthStore).token();
  if (token) {
    req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  }
  return next(req);
};
```

7. Role-Based UI Access Control

Feature / Action	Allowed Roles
Book a Session	ROLE_LEARNER
Accept / Reject Session	ROLE_MENTOR
Apply as Mentor	ROLE_LEARNER

Approve Mentor Application	ROLE_ADMIN
Create Learning Group	ROLE_LEARNER, ROLE_MENTOR
Manage Platform Users	ROLE_ADMIN
Submit Review & Rating	ROLE_LEARNER
View Platform Analytics	ROLE_ADMIN

8. Non-Functional Implementation

8.1 Performance

- Lazy-loaded feature modules reduce initial bundle by ~65%
- OnPush change detection on all list/card components
- Angular @defer blocks for below-fold content
- Virtual scrolling (CdkVirtualScrollViewport) for long lists

8.2 Security

- JWT stored in memory (not localStorage) — prevents XSS
- HttpOnly cookies for refresh tokens
- Angular DomSanitizer used for all dynamic HTML bindings
- CSP headers set in Nginx production config

8.3 Accessibility

- All interactive elements have ARIA labels and roles
- Angular CDK FocusTrap in modals and dialogs
- Color contrast ratio ≥ 4.5:1 across all screens
- Full keyboard navigation support

8.4 Testing

Type	Tool & Target
Unit Tests	Jest — Services, Stores, Pipes (≥80% coverage)
Component Tests	Angular Testing Library (≥75%)
E2E Tests	Cypress — All critical user flows (100%)
Accessibility	axe-core + cypress-axe

Performance	Lighthouse CI (score ≥ 90)

9. Build & Deployment

Dockerfile — Multi-stage Angular 18 Build

```
# Stage 1: Build Angular app
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --legacy-peer-deps
COPY . .
RUN npm run build -- --configuration production

# Stage 2: Serve with Nginx
FROM nginx:1.25-alpine
COPY --from=builder /app/dist/skillsync/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

SkillSync — Frontend Case Study v1.0

Angular 18 • NgRx Signals • Spring 2025